Software Project Management · 2019-20

CSE 421 | Session 2019-20 | June 2023 final

Model study answers. Every listed option is addressed. Question titles are paraphrased; exact wording and original diagrams are in the source pages appended to this pack.

Printed header: June 2023, session 2019-20; 2 hours, 50 marks. Overlapping photographs are preserved. All 34 options are answered. ROI uses the average-annual-profit convention in your course materials; payback timing assumptions are explicit.

Quick revision

Four Ps: People, Product, Process, Project.

Risk exposure = probability × loss; expected monetary value = sum(probability × outcome).

EF = ES + duration; ES = maximum predecessor EF; float = LS − ES.

Net profit = sum of all net cash flows, including the negative initial investment.

Course ROI/ARR = (net profit / project life) / initial investment × 100%.

Interpolated payback = last negative year + unrecovered amount / next year cash flow, assuming uniform receipts.

Critical path for 3(b): B-E-G-I-L-M; duration 15 weeks.

1(a) | The four Ps

People: stakeholders and the team who do and use the work. Product: objectives, scope, requirements and constraints. Process: the lifecycle and engineering practices used to build it. Project: planning, estimation, scheduling, monitoring and control. Example: for a library portal, librarians and developers are the people; borrowing functions define the product; iterative delivery is the process; the schedule and budget belong to project management.

Reading: Introduction - https://mhrumi.github.io/Software-project-management/01-introduction.html

Revision answer: People, Product, Process and Project.

1(b) | Feasibility study

A feasibility study checks whether a proposed system is practical and worthwhile before committing to full development. Assess technical capability, economic costs and benefits, operational fit, schedule and relevant legal constraints. Its recommendation is to proceed, revise or reject the proposal.

Reading: Project evaluation - https://mhrumi.github.io/Software-project-management/02-project-evaluation.html

Revision answer: Check whether the proposal can and should be carried out.

1(c) | Contract

In this project context, a contract is an enforceable agreement between customer and supplier setting out their obligations. It specifies deliverables, acceptance, price/payment, schedule and how changes, ownership, disputes and termination are handled.

Reading: Contracts - https://mhrumi.github.io/Software-project-management/10-managing-contracts.html

Revision answer: An agreement defining enforceable customer and supplier obligations.

1(d) | Cost-benefit analysis

Identify the expected costs and benefits of each project option, quantify them where reasonable and compare them over a stated period. Include development, operation and maintenance, as well as expected savings or income. Discount future cash flows when using NPV; acknowledge benefits that cannot credibly be monetized.

Reading: Project evaluation - https://mhrumi.github.io/Software-project-management/02-project-evaluation.html

Revision answer: Compare expected project costs with expected benefits.

1(e) | Cost flow forecasting

The paper says cost flow forecasting. It means estimating when project expenditure will occur, based on scheduled activities and resource use, so funding is available when needed. If the intended term is cash-flow forecasting, include both incoming and outgoing cash and calculate their net balance for each period.

Reading: Resource allocation - https://mhrumi.github.io/Software-project-management/08-resource-allocation.html

Revision answer: Forecast the timing of costs; cash-flow forecasting includes receipts too.

1(f) | Critical path

The critical path is the longest-duration dependency path from project start to finish. It fixes the earliest completion date in the precedence model. Its activities have zero total float under the usual earliest-finish schedule. A delay on an unchanged critical path delays project completion unless time is recovered elsewhere.

Reading: Activity planning - https://mhrumi.github.io/Software-project-management/06-activity-planning.html

Revision answer: The longest-duration path, determining minimum project duration.

1(g) | Inputs to total software project cost

Estimate effort for each role and activity, multiply by the corresponding labor rate, and add hardware/software, services, facilities, travel/training and allocated overhead. Include the agreed contingency or risk allowance. Formula: total budget = sum(effort × labor rate) + nonlabor direct costs + overhead + contingency. State whether the estimate covers development only or the whole lifecycle; elapsed time alone is not effort.

Reading: Resource allocation - https://mhrumi.github.io/Software-project-management/08-resource-allocation.html

Revision answer: Effort, labor rates, nonlabor costs, overhead and contingency.

1(h) | Maslow's hierarchy of needs

From lower to higher: physiological needs, safety, belonging/social needs, esteem and self-actualization. Workplace examples are reasonable pay and rest, job security, team inclusion, recognition, and opportunities for growth. This is a motivational framework, not a rigid rule that every person follows in exactly the same order.

Reading: Managing people - https://mhrumi.github.io/Software-project-management/11-managing-people.html

Revision answer: Physiological → Safety → Belonging → Esteem → Self-actualization.

2(a) | Why software projects are difficult

Invisibility: working progress is hard to see, so use demonstrable deliverables. Complexity: many interacting states and dependencies make prediction and testing difficult. Conformity: software must fit external rules, legacy systems and sometimes inconsistent requirements. Flexibility: because software can be changed, stakeholders may expect unlimited change; assess and control its effects on scope, cost and time.

Reading: Introduction - https://mhrumi.github.io/Software-project-management/01-introduction.html

Revision answer: Invisibility, complexity, conformity and flexibility.

2(b) | Project portfolio management

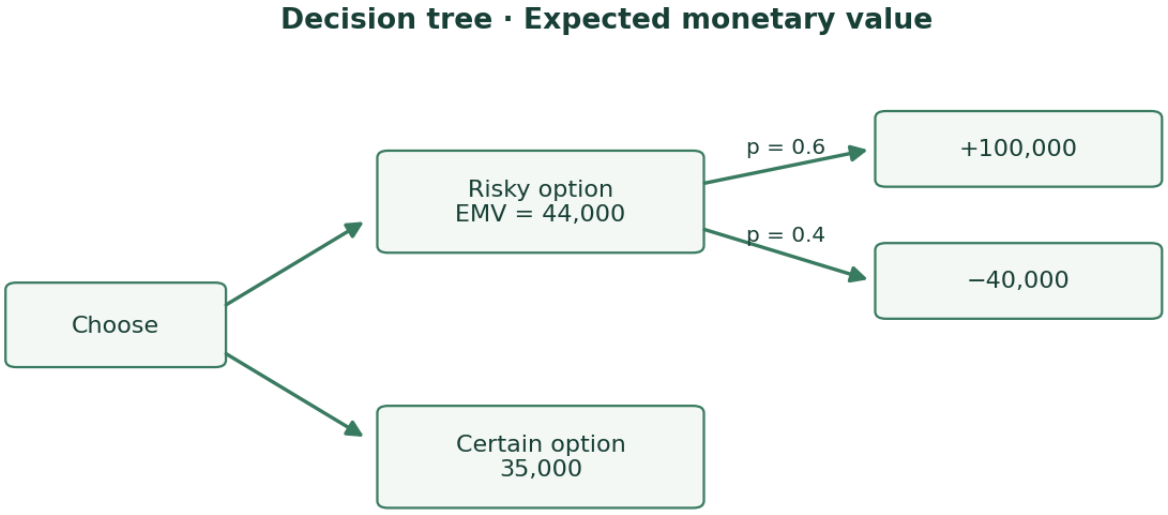
Portfolio management coordinates a set of projects so limited resources support organizational strategy. Portfolio definition establishes what work exists or is proposed, including unofficial work. Portfolio management prioritizes, resources, monitors and coordinates that work. Portfolio optimization adjusts the mix to balance strategic value, risk, capacity and short- versus long-term benefits; stop or defer weak proposals when justified.

Reading: Project evaluation - https://mhrumi.github.io/Software-project-management/02-project-evaluation.html

Revision answer: Define the portfolio, manage it, then optimize its balance.

2(c) | Decision trees for risk evaluation

Represent a choice as a decision node and its uncertain outcomes as chance branches. Assign probabilities and monetary outcomes, including relevant costs. Calculate EMV = sum(p_i × outcome_i) at each chance node and work backward to choose the preferable option. Example: a risky option gives +100,000 with probability 0.6 and −40,000 with probability 0.4, so EMV = 60,000 −16,000 = 44,000. A certain +35,000 has lower EMV, but a risk-averse decision maker may still prefer it. EMV is not a guaranteed outcome.



Reading: Project evaluation - https://mhrumi.github.io/Software-project-management/02-project-evaluation.html

Revision answer: Compare probability-weighted outcomes by rolling back a decision tree.

2(d) | Selecting the right person for a job

Define the role, tasks and essential competencies first. Write a job description and a person specification with justified selection criteria. Shortlist against those criteria, use a structured interview plus a relevant work sample, and check evidence/references with appropriate consent. Consider communication, teamwork, learning ability and availability alongside technical skills. Record a fair comparison, select the best fit and arrange induction/training; do not rely only on personal impressions.

Reading: Managing people - https://mhrumi.github.io/Software-project-management/11-managing-people.html

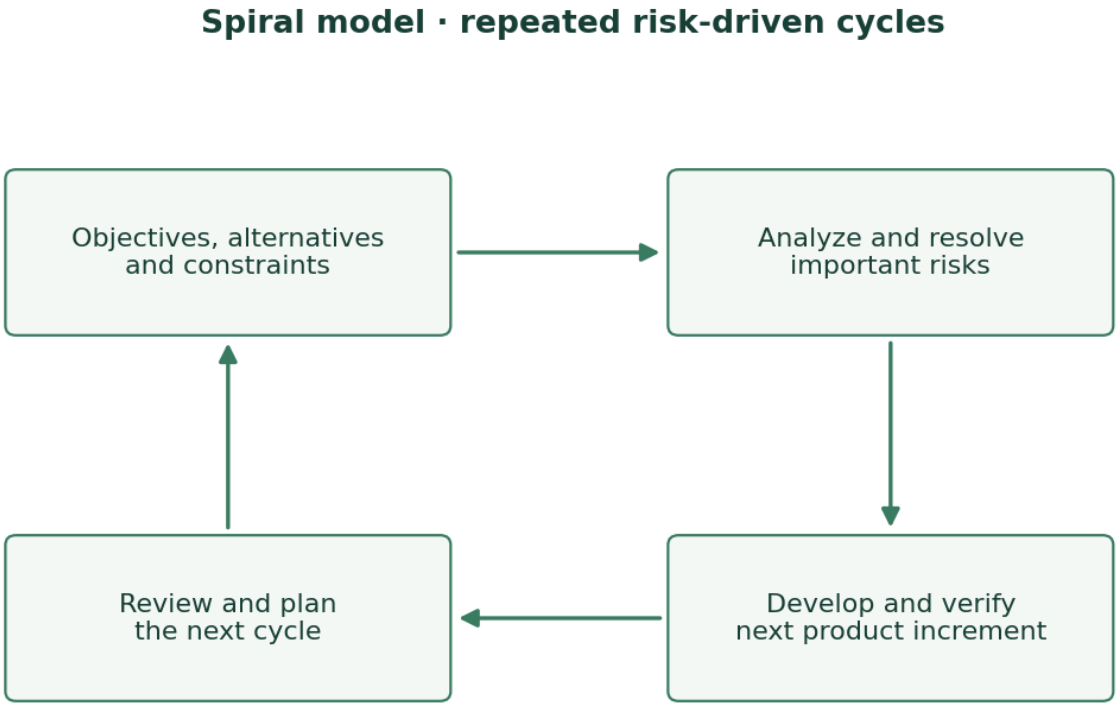
Revision answer: Define requirements, gather job-relevant evidence, compare fairly and onboard.

2(e) | Waterfall drawbacks and the spiral model

Waterfall proceeds through requirements, design, implementation, testing and delivery. Its weaknesses include expensive late changes, late user feedback and integration/testing problems discovered after substantial work. It suits stable, well-understood requirements better than highly uncertain ones.

The spiral model repeats a risk-driven cycle: establish objectives, alternatives and constraints; evaluate alternatives and resolve major risks, often through prototypes; develop and verify the next product level; review results and plan the next cycle.

Merits: early attention to high risks, repeated stakeholder feedback, incremental commitment and accommodation of change. Demerits: risk analysis needs expertise, management overhead can be high, completion can be difficult to bound and it may be excessive for small low-risk work.



Reading: Project approach - https://mhrumi.github.io/Software-project-management/04-project-approach.html

Revision answer: Waterfall handles change poorly; spiral uses repeated risk-driven cycles.

2(f) | Managing activity risks

For each work package, identify what could prevent its completion: uncertain estimates, skills gaps, unavailable resources or dependency failure. Assess probability and schedule/cost impact; prioritize risks, especially on critical or near-critical paths. Assign an owner and choose avoidance, reduction, transfer or acceptance. Add justified contingency and trigger-based response actions to the plan. Review the risk log and revise estimates/dependencies as evidence changes. Example: prototype a risky migration early, retain a recovery plan and reserve time for data cleanup.

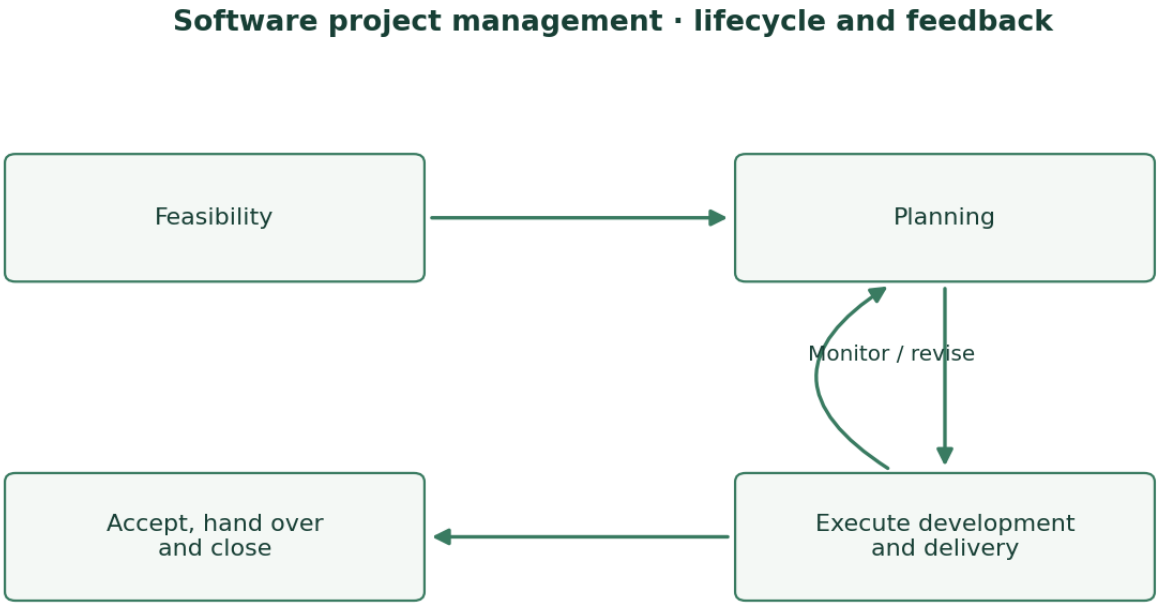
Reading: Risk management - https://mhrumi.github.io/Software-project-management/07-risk-management.html

Revision answer: Identify, assess, assign, respond, reserve and monitor.

3(a) | Activities covered by software project management

Feasibility study: establish the business need, possible solutions, expected benefits, costs and viability; decide whether to proceed. Planning: specify objectives, scope, products, activities, dependencies, resources, estimates, schedule, quality and risk responses. Project execution: coordinate development and delivery while monitoring actual progress, controlling changes and corrective action, and managing people/contracts. Close with acceptance, handover and lessons.

The diagram separates the management lifecycle from development tasks: requirements, design, implementation and testing happen within execution. Monitoring and control provide feedback that can revise the plan throughout the project.



Reading: Introduction - https://mhrumi.github.io/Software-project-management/01-introduction.html

Revision answer: Feasibility → Planning → Execution, with monitoring/control feedback and closure.

3(b) | Precedence diagram and critical path

Step 1: draw one node per activity and connect each predecessor to its successor. Start A and B together; finish only after M. Activity durations are in weeks.

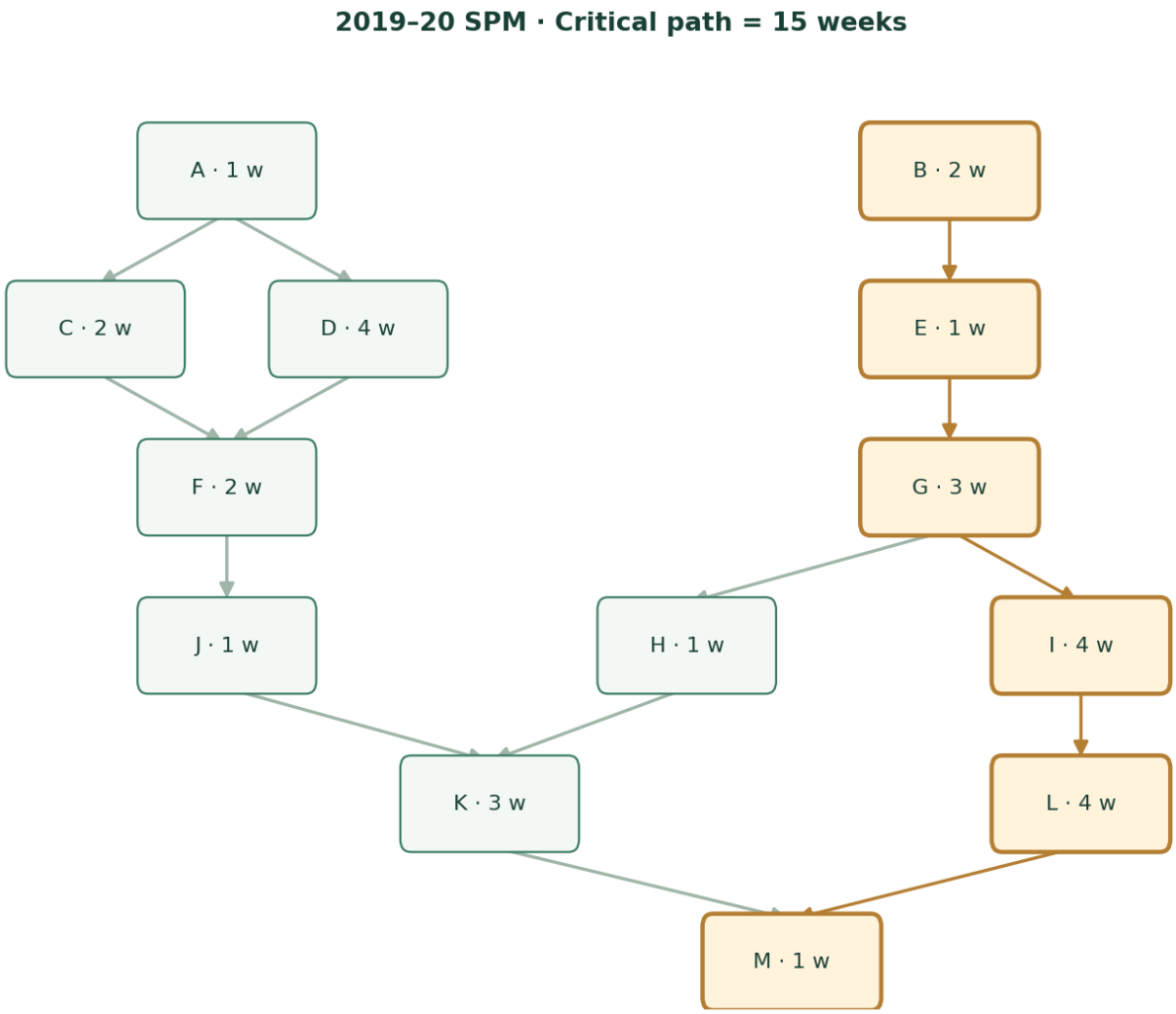
Step 2, forward pass: ES = maximum predecessor EF; EF = ES + duration. For example F waits for both C and D: ES(F)=max(3,5)=5, EF(F)=7. M waits for K and L: ES(M)=max(11,14)=14, EF(M)=15.

Step 3, backward pass: set finish=15; LF = minimum successor LS, LS=LF−duration; float=LS−ES.

Activity	duration	predecessors	ES	EF	LS	LF	float
A	1	-	0	1	3	4	3
B	2	-	0	2	0	2	0
C	2	A	1	3	6	8	5
D	4	A	1	5	4	8	3
E	1	B	2	3	2	3	0
F	2	C,D	5	7	8	10	3
G	3	E	3	6	3	6	0
H	1	G	6	7	10	11	4
I	4	G	6	10	6	10	0

Activity	duration	predecessors	ES	EF	LS	LF	float
J	1	F	7	8	10	11	3
K	3	J,H	8	11	11	14	3
L	4	I	10	14	10	14	0
M	1	K,L	14	15	14	15	0

Step 4: trace the connected zero-float activities B-E-G-I-L-M. Duration=2+1+3+4+4+1=15 weeks. A-D-F-J-K-M takes 12 weeks, so it does not control completion. Critical path assumes the printed dependencies and unlimited resource availability; resource constraints can change the schedule.



Reading: Activity planning - https://mhrumi.github.io/Software-project-management/06-activity-planning.html

Revision answer: Critical path B-E-G-I-L-M; total duration 15 weeks.

3(c) | Stages of contract placement

1. Analyse requirements and define the required deliverables, quality and acceptance criteria. 2. Prepare an invitation to tender or request for proposals with scope, deadlines and evaluation criteria. 3. Identify suitable suppliers, issue the invitation and manage clarifications fairly. 4. Evaluate proposals using the stated technical, commercial, capability and risk criteria; shortlist and check evidence, demonstrations or references. 5. Negotiate scope, responsibilities, acceptance, payment, intellectual property and change procedures. 6. Award and sign the contract, notify bidders appropriately and hand it to contract management for monitoring. Placement ends with the agreement; ongoing supplier monitoring follows it.

Reading: Contracts - https://mhrumi.github.io/Software-project-management/10-managing-contracts.html

Revision answer: Requirements → invitation → supplier proposals → evaluation → negotiation → award.

4(a) | Simplest software development paradigm

Build-and-fix is the simplest informal paradigm: write an initial version, inspect it and repeatedly modify it until it seems satisfactory. It has little explicit requirements analysis, design or planning. This can be tolerable for a tiny personal experiment but gives weak control over larger projects. Waterfall is a simple structured sequential lifecycle; it is distinct from build-and-fix, which appears first in your course's approach lesson.

Reading: Project approach - https://mhrumi.github.io/Software-project-management/04-project-approach.html

Revision answer: Build-and-fix: build an initial version, then repeatedly repair it.

4(b) | PRINCE2

PRINCE2 expands to PRojects IN Controlled Environments, version 2. It is a structured project-management method emphasizing continued business justification, defined roles, staged management and control by exception. It manages the project and can coexist with different software development lifecycles.

Reading: Step Wise planning - https://mhrumi.github.io/Software-project-management/03-project-planning.html

Revision answer: A structured, business-justified, stage-based project-management method.

4(c) | Project stakeholders

Stakeholders are people, groups or organizations that affect the project, are affected by it or have an interest in its outcomes. Examples include sponsor, customer, end users, project team, supplier and regulator. Identify their needs, influence and communication requirements.

Reading: Introduction - https://mhrumi.github.io/Software-project-management/01-introduction.html

Revision answer: Those who affect, are affected by or have an interest in the project.

4(d) | Data mining for credit-card fraud detection

For unusual transactions without reliable fraud labels, use anomaly/outlier detection: flag transactions that deviate from normal behavior. When historical transactions have fraud/nonfraud labels, supervised classification can estimate fraud risk. For this short question, anomaly detection is the usual expected task; the correct modeling choice depends on available labels.

Revision answer: Anomaly detection; supervised classification is suitable when labels exist.

4(e) | Bespoke system

A bespoke system is designed or adapted specifically for one customer's requirements, rather than sold as an unchanged general-purpose package. Example: custom admission-processing software that implements a university's exact rules and integrations.

Reading: Introduction - https://mhrumi.github.io/Software-project-management/01-introduction.html

Revision answer: Custom-built software for a particular customer.

4(f) | Requirements analysis

Requirements analysis examines stakeholder needs to clarify, organize, model, check and agree what the system must do and the constraints it must satisfy. Resolve conflicts and ambiguities, prioritize requirements and define verifiable acceptance criteria. It includes functional and nonfunctional needs.

Reading: Introduction - https://mhrumi.github.io/Software-project-management/01-introduction.html

Revision answer: Clarify and validate agreed functional needs and quality constraints.

4(g) | Work Breakdown Structure

A WBS hierarchically decomposes the complete project scope into deliverables and manageable work packages. It supports estimation, ownership and control. The child elements should collectively cover the parent scope without overlap. Dependencies and dates belong in the associated schedule, not in the WBS alone.

Reading: Activity planning - https://mhrumi.github.io/Software-project-management/06-activity-planning.html

Revision answer: A hierarchical decomposition of all project work.

4(h) | Payback period

The payback period is the time required for accumulated net cash inflows to recover the initial investment. It measures recovery speed, not total lifetime profit. Ordinary payback ignores the time value of money and cash flows after recovery; discounted payback is a different measure.

Reading: Project evaluation - https://mhrumi.github.io/Software-project-management/02-project-evaluation.html

Revision answer: Time to recover the original investment from net cash inflows.

5(a) | Egoless programming

Egoless programming separates a programmer's self-worth from their code. Treat defects as issues to improve, welcome peer review, share knowledge and let the team own the product. Critique the work respectfully rather than blame the author; accept evidence and revise a design when a better one is found. It encourages review and collective learning without eliminating individual accountability.

Reading: Teams and leadership - https://mhrumi.github.io/Software-project-management/12-working-in-teams.html

Revision answer: Review code without personal defensiveness; improve it collaboratively.

5(b) | Hackman-Oldham job characteristics model

The paper's "Oldhun-Hackman" refers to Hackman and Oldham. Five core job dimensions are skill variety, task identity, task significance, autonomy and feedback. The first three support experienced meaningfulness; autonomy supports responsibility; feedback gives knowledge of results. These psychological states can promote intrinsic motivation, satisfaction and effective work, moderated by individual and contextual differences.

Motivating Potential Score: MPS=[(skill variety + task identity + task significance)/3] × autonomy × feedback, using compatible rating scales. Example: give a developer ownership of a complete user-facing feature, freedom over implementation within constraints, and direct feedback about how it works.

The supplied Week 11 page does not explicitly cover this model. Supplemental primary reference: Hackman & Oldham, Motivation through the design of work: test of a theory (1976), DOI 10.1016/0030-5073(76)90016-7.

Reading: Hackman & Oldham (1976), original study - https://doi.org/10.1016/0030-5073(76)90016-7

Revision answer: Five dimensions support meaningfulness, responsibility and knowledge of results.

5(c) | Objectives of activity planning

Identify the work needed to deliver the agreed products; establish dependencies and a feasible sequence; estimate duration and resource needs; allocate responsibility; identify critical activities and risks; and create a schedule/baseline for monitoring and control. It also checks whether deadlines and resource limits are achievable before committing to them.

Reading: Activity planning - https://mhrumi.github.io/Software-project-management/06-activity-planning.html

Revision answer: Identify, sequence, estimate and resource work, then establish a controllable schedule.

5(d) | Factors making a job meaningful

In the Hackman-Oldham model, experienced meaningfulness is supported by skill variety (using different skills), task identity (completing an identifiable whole piece of work) and task significance (making a worthwhile difference to others). For example, building and validating an entire accessible booking feature uses several skills, has a visible result and helps real users. Autonomy and feedback matter too, but map primarily to responsibility and knowledge of results.

Revision answer: Skill variety, task identity and task significance.

5(e) | Fixed-price contract: advantages and disadvantages

The supplier agrees a price for a defined scope, subject to the agreed change terms. Customer advantages: greater initial budget predictability, easier bid comparison and more cost-overrun risk allocated to the supplier. Supplier advantages: clear commercial target and potential gain from efficiency.

Disadvantages: a precise scope is needed; uncertainty may produce a large risk premium; changes require negotiation and can trigger disputes; a supplier under pressure may cut corners unless quality and acceptance are explicit. The supplier bears estimation/overrun risk. A fixed price does not mean every later scope change is free.

Reading: Contracts - https://mhrumi.github.io/Software-project-management/10-managing-contracts.html

Revision answer: Budget clarity and efficiency incentives, but scope rigidity, risk premiums and change disputes.

5(f) | Risk assessment and formula

Risk assessment identifies and analyzes uncertain events to prioritize response. A quantitative expected-loss measure is risk exposure RE=P(event) × loss if it occurs. Example: probability 0.2 and loss 50,000 gives RE=10,000 in expected monetary loss. For a probability-impact matrix, ordinal category scores can be multiplied for prioritization, but the result is a ranking score rather than a literal currency expectation. Also inspect high-impact risks that a simple average can obscure.

Reading: Risk management - https://mhrumi.github.io/Software-project-management/07-risk-management.html

Revision answer: Risk exposure = probability × consequence (expected loss when quantified).

6(a) | Leadership models and leader functions

Style models distinguish autocratic/directive decisions, democratic/participative decisions and laissez-faire/delegated working. Their suitability depends on urgency, competence and coordination needs; delegation still requires accountability.

The course's situational model tailors the approach to the specific task: directing gives a novice clear steps; coaching adds explanation and feedback as competence grows; supporting shares decisions when capability is high but confidence/motivation needs help; delegating gives capable, committed people responsibility within agreed goals.

Leader functions include setting direction, organizing responsibilities, communicating priorities, motivating and developing people, resolving conflict, removing obstacles, representing the team and monitoring outcomes. Example: during a first production release, guide the junior through a checklist, pair them with an experienced colleague and progressively delegate as they demonstrate competence.

Reading: Teams and leadership - https://mhrumi.github.io/Software-project-management/12-working-in-teams.html

Revision answer: Choose a suitable style; provide direction, support, coordination and accountability.

6(b) | Step Wise project planning

0. Select the project: confirm business justification. 1. Identify scope and objectives: state deliverables and success criteria. 2. Identify project infrastructure: standards, roles, tools and reporting. 3. Analyze project characteristics: choose an appropriate development approach. 4. Identify products and activities: define deliverables, work and dependencies. 5. Estimate activity effort/duration. 6. Identify activity risks and response needs. 7. Allocate resources and produce a feasible schedule. 8. Review and publicize the plan with stakeholder agreement. 9. Execute the plan and monitor it. 10. Perform lower-level planning as activities approach, revisiting products, estimates and risks.

Planning is iterative: insufficient resources can require revised estimates, scope or sequencing, followed by review and approval. Your course diagram separates execution and lower-level planning, while its prose combines them under step 9; both operations are included here.

Reading: Step Wise planning - https://mhrumi.github.io/Software-project-management/03-project-planning.html

Revision answer: Select → objectives → infrastructure → characteristics → products/activities → estimates → risks → resources → review → execute and replan.

6(c) | Net profit, payback and ROI: all four projects

Use the printed question table, not a similar textbook table: Project 1 year 3 is 20,000, and Project 3 year 5 is 20,000. All values use the table's unstated currency unit.

Step 1: net profit = total positive inflows − initial investment. Step 2: compute cumulative cash flow until investment is recovered. Step 3: use the course's annual ROI/ARR = (net profit / 5) / initial investment × 100%.

Project 1: inflows=160,000; net profit=160,000−100,000=60,000. Cumulative at years 0-5: -100,000, -90,000, -80,000, -60,000, -40,000, 60,000. Payback=4+40,000/100,000=4.4000 years. Average annual profit=60,000/5=12,000; ROI=12,000/100,000×100=12.00%.

Project 2: inflows=1,150,000; net profit=1,150,000−1,000,000=150,000. Cumulative at years 0-5: -1,000,000, -800,000, -600,000, -400,000, -200,000, 150,000. Payback=4+200,000/350,000=4.5714 years. Average annual profit=150,000/5=30,000; ROI=30,000/1,000,000×100=3.00%.

Project 3: inflows=130,000; net profit=130,000−100,000=30,000. Cumulative at years 0-5: -100,000, -70,000, -40,000, -10,000, 10,000, 30,000. Payback=3+10,000/20,000=3.5000 years. Average annual profit=30,000/5=6,000; ROI=6,000/100,000×100=6.00%.

Project 4: inflows=165,000; net profit=165,000−120,000=45,000. Cumulative at years 0-5: -120,000, -90,000, -60,000, -30,000, -5,000, 45,000. Payback=4+5,000/50,000=4.1000 years. Average annual profit=45,000/5=9,000; ROI=9,000/120,000×100=7.50%.

Summary: Project 1: profit 60,000; payback 4.4000 years; annual ROI 12%. Project 2: profit 150,000; payback 4.5714 years; annual ROI 3%. Project 3: profit 30,000; payback 3.5000 years; annual ROI 6%. Project 4: profit 45,000; payback 4.1000 years; annual ROI 7.5%.

Payback interpolation assumes the next year's inflow arrives evenly. If all annual flows occur only at each year-end, payback is at year 5 for Projects 1, 2 and 4, and year 4 for Project 3. The materials use interpolation, so that is the main answer.

Cumulative lifetime return net profit/investment is 60%, 15%, 30% and 37.5%, respectively; these are not the annual ROI values requested under the course convention. Highest absolute profit: Project 2. Fastest payback: Project 3. Highest annual ROI: Project 1.

Reading: Project evaluation - https://mhrumi.github.io/Software-project-management/02-project-evaluation.html

Revision answer: P1: 60,000; 4.4 y; 12%. P2: 150,000; 4.5714 y; 3%. P3: 30,000; 3.5 y; 6%. P4: 45,000; 4.1 y; 7.5%.

Reference reading

User-provided SPM course materials: https://mhrumi.github.io/Software-project-management/

Bob Hughes and Mike Cotterell, Software Project Management.

Roger S. Pressman and Bruce R. Maxim, Software Engineering: A Practitioner's Approach.

Barry W. Boehm, Software Engineering Economics.

Steve McConnell, Rapid Development.

ISO 9001:2015 (historical exam context): https://www.iso.org/standard/62085.html

Introduction - https://mhrumi.github.io/Software-project-management/01-introduction.html

Project evaluation - https://mhrumi.github.io/Software-project-management/02-project-evaluation.html

Step Wise planning - https://mhrumi.github.io/Software-project-management/03-project-planning.html

Project approach - https://mhrumi.github.io/Software-project-management/04-project-approach.html

Estimation concepts - https://mhrumi.github.io/Software-project-management/05a-estimation-concepts.html

Estimation models - https://mhrumi.github.io/Software-project-management/05b-estimation-models.html

Activity planning - https://mhrumi.github.io/Software-project-management/06-activity-planning.html

Risk management - https://mhrumi.github.io/Software-project-management/07-risk-management.html

Resource allocation - https://mhrumi.github.io/Software-project-management/08-resource-allocation.html

Monitoring and control - https://mhrumi.github.io/Software-project-management/09-monitoring-control.html

Contracts - https://mhrumi.github.io/Software-project-management/10-managing-contracts.html

Managing people - https://mhrumi.github.io/Software-project-management/11-managing-people.html

Teams and leadership - https://mhrumi.github.io/Software-project-management/12-working-in-teams.html

Software quality - https://mhrumi.github.io/Software-project-management/13-software-quality.html

These are supporting reading references, not claims that this paper or its answer key appears in those books. Course-specific terminology and source ambiguities are noted beside the affected answers.

Shahjalal University of Science and Technology
Department of Computer Science and Engineering
4th Year 1st Semester Final Examination' Jun 2023 (Session: 2019-20)
Course Code: CSE 421 Credits: 2 Course Title: Software Project Management
Time: 2 hrs Total Marks: 50

Group A

[Answer all the questions]

1. Write any FIVE **5x1=5**

a) What are the four P's that effective software project management focuses on?
b) Define Feasibility Study.
c) What is a contract?
d) What is the cost-benefit analysis?
e) What is meant by cost flow forecasting?
f) What is a Critical path?
g) What parameters are involved in computing the total cost of a software development project?
h) What is Masslow's hierarchy needs?

2. Answer any FOUR **4x2.5=10**

a) What are the characteristics of software projects that make them more difficult than other projects?
b) What is project portfolio management? Explain the key aspects of project portfolio management.
c) What is the use of decision tree in risk evaluation?
d) How would you select the right person for a job?
e) Explain the drawback of the waterfall model. Also, discuss the spiral model with its merits and demerits.
f) How to manage activity risks?

3. Answer any TWO **2x5=10**

a) What are the activities covered by software project management? Explain with a diagram.
b) Construct a precedence diagram to analyze your project. Show the critical path on your precedence diagram.

Activity	Duration (week)	Predecessors

on?

b) Define Feasibility Study.

c) What is a contract?

d) What is the cost-benefit analysis?

e) What is meant by cost flow forecasting?

f) What is a Critical path?

g) What parameters are involved in computing the total cost of a software development project?

h) What is Masslow's hierarchy needs?

4x2.5=10

2. Answer any FOUR

a) What are the characteristics of software projects that make them more difficult than other projects?

b) What is project portfolio management? Explain the key aspects of project portfolio management.

c) What is the use of decision tree in risk evaluation?

d) How would you select the right person for a job?

e) Explain the drawback of the waterfall model. Also, discuss the spiral model with its merits and demerits.

f) How to manage activity risks?

2x5=10

3. Answer any TWO

a) What are the activities covered by software project management? Explain with a diagram.

b) Construct a precedence diagram to analyze your project. Show the critical path on your precedence diagram.

Activity	Duration (week)	Predecessors
A	1	-
B	2	-
C	2	A
D	4	A
E	1	B
F	2	C, D
G	3	E
H	1	G
I	4	G
J	1	F
K	3	J, H
L	4	I
M	1	K, L

c) Briefly describe the stages of a contract placement

Group B

[Answer all the questions]

4. Write short notes (Any FIVE) **5x1=5**

a) What is the simplest model of software development paradigm?
b) What is PRINCE2?
c) Define project stakeholders.
d) To detect fraudulent usage of credit cards, what data mining task should be used?
e) What is a bespoke system?
f) What is requirement Analysis?
g) What is Work Breakdown structure?
h) What is the payback period?

5. Answer any FOUR **4x2.5=10**

a) What do you understand by Egoless Programming?
b) Explain the Oldhun-Hackman job characteristic model.
c) What are the objectives of activity planning?
d) What are the factors that made the job meaningful?
e) What are the advantages & disadvantages of fixed price contracts?
f) Define risk assessment using formula.

6. Answer any TWO **2x5=10**

a) Discuss leadership models. Explain function of a leader with an example
b) Briefly explain about Stepwise project planning.
c) Based on the data given below. Calculate Net Profit, Payback, and ROI for each project

Year	Project 1	Project 2	Project 3	Project 4
0	-100,000	-1,000,000	-100,000	-120,000
1	10,000	200,000	30,000	30,000
2	10,000	200,000	30,000	30,000

Group B

[Answer all the questions]

4. Write short notes (Any FIVE) **5x1=5**

a) What is the simplest model of software development paradigm?
b) What is PRINCE2?
c) Define project stakeholders.
d) To detect fraudulent usage of credit cards, what data mining task should be used?
e) What is a bespoke system?
f) What is requirement Analysis?
g) What is Work Breakdown structure?
h) What is the payback period?

5. Answer any FOUR **4x2.5=10**

a) What do you understand by Egoless Programming?
b) Explain the Oldhun-Hackman job characteristic model.
c) What are the objectives of activity planning?
d) What are the factors that made the job meaningful?
e) What are the advantages & disadvantages of fixed price contracts?
f) Define risk assessment using formula.

6. Answer any TWO **2x5=10**

a) Discuss leadership models. Explain function of a leader with an example
b) Briefly explain about Stepwise project planning.
c) Based on the data given below. Calculate Net Profit, Payback, and ROI for each project

Year	Project 1	Project 2	Project 3	Project 4
0	-100,000	-1,000,000	-100,000	-120,000
1	10,000	200,000	30,000	30,000
2	10,000	200,000	30,000	30,000
3	20,000	200,000	30,000	30,000
4	20,000	200,000	20,000	25,000
5	100,000	350,000	20,000	50,000